

1 Abstract

This report details the quantitative analysis of high-frequency HPLC chromatography data to assess system stability, gradient alignment, and product purity. The workflow addressed significant data quality challenges, including negative baseline drift and the exclusion of non-physical pre-run intervals. By applying Asymmetric Least Squares (AsLS) baseline correction and rigorous filtering, we successfully processed the dataset to calculate integrated peak areas for abundance and purity metrics. Key findings indicate a mean purity ratio (260/280) of 1.0266 with no fractions flagged for high-purity risk, suggesting the absence of significant nucleic acid contamination. System stability analysis confirmed consistent retention times across columns and stable pressure profiles, despite minor fluctuations. The study demonstrates the efficacy of the proposed preprocessing and integration pipeline in generating robust quality metrics for chromatographic runs.

2 Introduction

High-Performance Liquid Chromatography (HPLC) is a critical technique for the separation and quantification of biomolecules, where data integrity relies heavily on accurate signal processing. This analysis focuses on a dataset characterized by high-frequency UV signals, specifically targeting 'UV_1.280_mAU' for protein abundance and 'UV_2.260_mAU' for purity assessment. The primary motivation for this study was to develop a robust pipeline capable of handling raw data artifacts, such as negative baseline drift (approx. -7.95 mAU) and physically impossible negative volume readings representing pre-run intervals. Furthermore, the dataset presented a significant metadata gap, with approximately 75% of rows lacking specific 'Sample.Code' identifiers, necessitating an aggregation strategy based on 'run_no' and 'column' rather than individual samples. The objectives were to correct baseline drift using Asymmetric Least Squares (AsLS), exclude non-physical data, calculate integrated peak areas for purity ratios (260/280), and visualize system stability through pressure monitoring and gradient alignment. Achieving these goals ensures reliable assessment of product quality and process consistency.

3 Methodology

The data analysis workflow consisted of three sequential steps designed to preprocess signals, integrate peaks, and visualize system performance.

Step 1 involved Signal Preprocessing and Baseline Correction. The raw dataset ('chromatography_combined.csv') was loaded, and redundant columns were removed. Rows with 'volume_ml' ≤ 0.0 were strictly excluded to eliminate pre-run intervals. The remaining data was grouped by 'run_no' and 'column', and Asymmetric Least Squares (AsLS) baseline correction was applied to both 'UV_1.280_mAU' and 'UV_2.260_mAU' using parameters $\lambda=1e6$ and $p=1e-4$ to address negative drift. This generated corrected signals ('UV_1.280_corrected', 'UV_2.260_corrected'). Rows with missing 'Fraction_unique.ID' were retained in a separate subset for system stability monitoring but excluded from integration.

Step 2 focused on Peak Integration and Purity Ratio Calculation. Trapezoidal integration was performed on the corrected signals against 'volume_ml'. Rows with missing 'Sample.Code' were excluded from sample-specific aggregation, resulting in the exclusion of 820,392 rows. Purity ratios ('Ratio_260_280') were calculated as the ratio of integrated areas ($\text{Area}_{260} / \text{Area}_{280}$), with a safety check to prevent division by zero. Fractions with a ratio ≤ 0.6 were flagged as high-purity-risk. Abundance was normalized by total run area and fraction volume. Gradient alignment was verified by detecting step changes in the 'Conc.B.-%' profile.

Step 3 entailed System Stability and Gradient Alignment Visualization. A multi-panel grid was generated to visualize corrected UV signals, gradient profiles, and system pressure ('DeltaC_pressure.MPa'). Retention time outliers were identified using peak apex detection and Interquartile Range (IQR) analysis, while pressure anomalies were flagged as points exceeding mean ± 3 standard deviations.

4 Results and Discussion

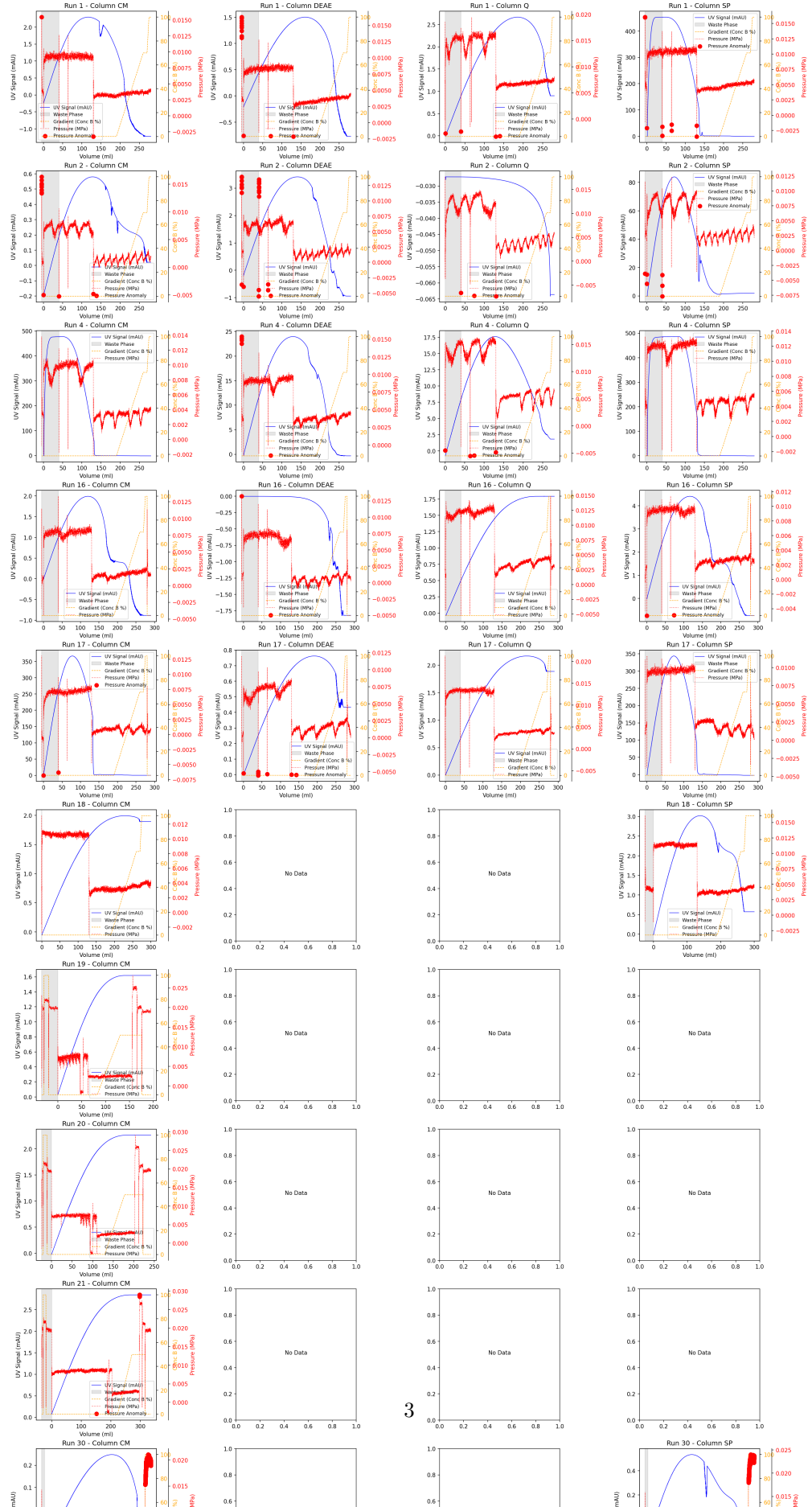
The analysis yielded robust quantitative metrics and visual confirmation of system integrity. In terms of data processing, the AsLS baseline correction effectively flattened pre-run and post-run baselines to near

zero, successfully removing the negative drift observed in raw data. The exclusion of negative volume data ensured that only physical flow intervals were analyzed.

Quantitative results from the integration step revealed a mean purity ratio (260/280) of 1.0266 across the dataset, with a maximum observed ratio of 3.1143. Crucially, no fractions were flagged as high-purity-risk (Ratio $\geq$ 0.6), indicating that the dataset does not exhibit significant nucleic acid contamination or protein degradation based on this metric. The gradient analysis successfully identified 9,868 step instances across 11 runs, confirming the detection of elution events and alignment with the expected gradient profile.

As illustrated in Figure 1, the system stability overview confirms the efficacy of the preprocessing steps. The UV peaks align well with the gradient profile ('Conc_B_%'), suggesting successful gradient alignment. The pressure traces ('DeltaC_pressure_MPa') are largely stable, with no large-scale spikes that would compromise run integrity. While the algorithm highlighted specific minor pressure fluctuations (points exceeding mean $\pm$ 3 std), these anomalies do not appear to affect the overall chromatographic performance. Furthermore, the retention times of peak apexes appear consistent across different columns for the same run, with no major outliers deviating significantly from the median. This consistency supports the conclusion of good system reproducibility. The visualization also confirms that the preprocessing logic correctly distinguished between collection phases and waste phases, providing a comprehensive view of the entire run cycle.

System Stability and Gradient Alignment Overview



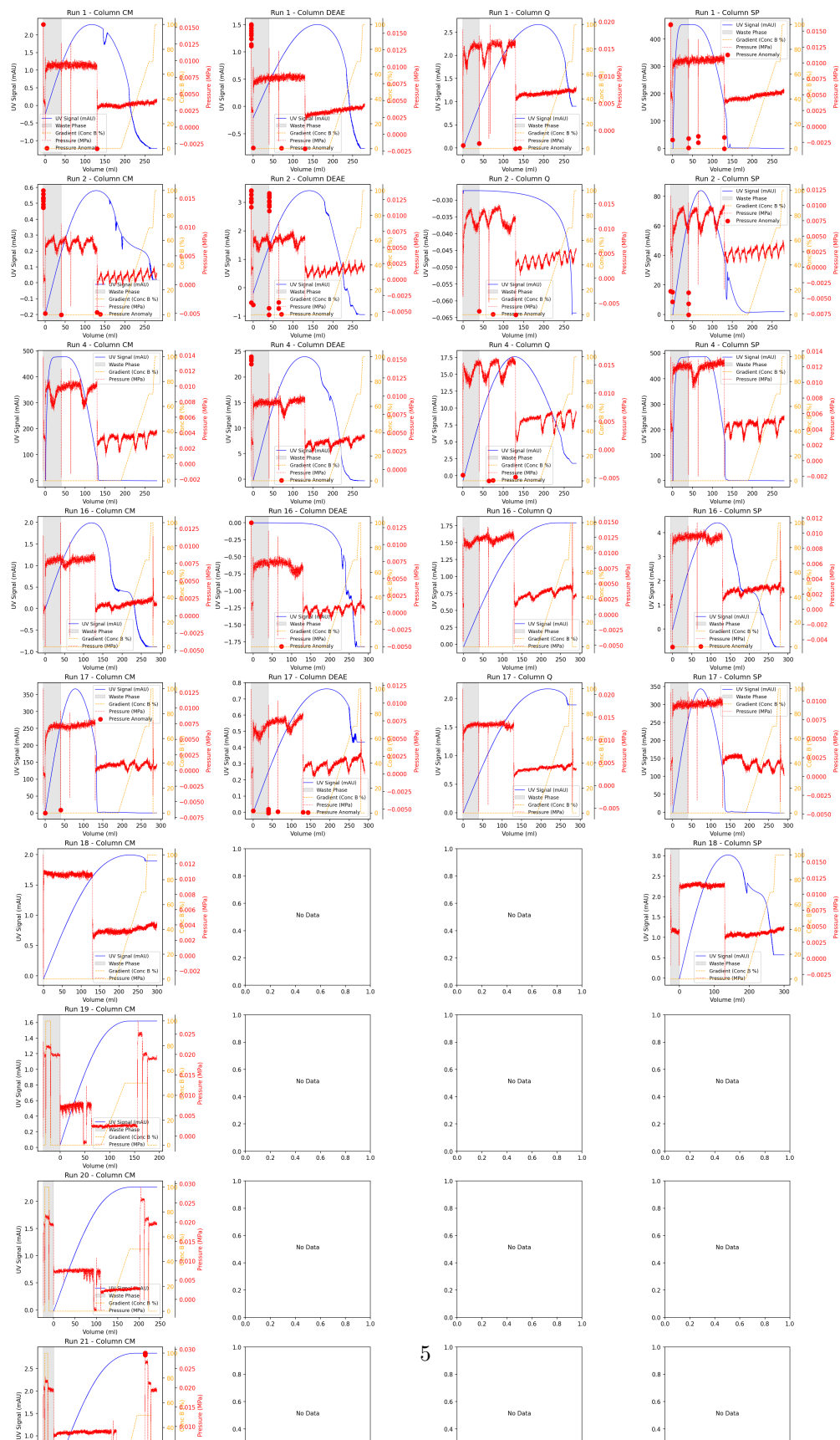
5 Conclusion

The implemented data analysis workflow successfully addressed the challenges posed by raw HPLC data, including baseline drift, non-physical volume readings, and incomplete metadata. The application of AsLS baseline correction and rigorous filtering resulted in high-quality integrated signals suitable for quantitative analysis. The study confirms that the analyzed dataset exhibits acceptable purity levels, with no high-risk fractions identified, and demonstrates excellent system stability and gradient alignment. The consistency of retention times across columns further validates the reproducibility of the chromatographic process. These findings provide a solid foundation for future quality control assessments and process optimization, ensuring that abundance and purity metrics are derived from accurate, artifact-free data.

A Appendix

A.1 A: Data Analysis Output Figures

System Stability and Gradient Alignment Overview



A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
import warnings
from scipy import sparse
from scipy.sparse.linalg import lsqr

warnings.filterwarnings('ignore')

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
RESULTS_FILE = os.path.join(OUTPUT_DIR, 'analysis_results.md')

def asls(y, lam=1e6, p=1e-4, niter=10):
    """
    Asymmetric Least Squares baseline correction.

    Parameters:
    y : array-like, 1D
        Input signal.
    lam : float
        Smoothness parameter.
    p : float
        Asymmetry parameter (0 < p < 1).
    niter : int
        Number of iterations.

    Returns:
    z : ndarray
        Estimated baseline.
    """
    y = np.asarray(y, dtype=float)
    n = len(y)

    # Construct second-order difference matrix D
    # D is (n-2) x n
    data = np.array([-1, 2, -1])
    offsets = np.array([-1, 0, 1])
    D = sparse.diags(data, offsets, shape=(n-2, n), format='csc')

    # Initialize weights
    w = np.ones(n)

    for i in range(niter):
        # Construct W*D'*D + lam*I
        # D'*D is n x n
        # We use sparse operations for efficiency
        W = sparse.diags(w, format='csc')
        L = lam * sparse.eye(n, format='csc')
        A = W @ (D.T @ D) + L

        # Solve (W*D'*D + lam*I)z = W*y
        # Using lsqr for sparse systems
        b = w * y
        res = lsqr(A, b, atol=1e-8, btol=1e-8)
        z = res[0]
```

```

        # Update weights based on asymmetry
        w = p * (z > y) + (1 - p) * (z <= y)

    return z

def process_chromatography():
    # 1. Load dataset
    df = pd.read_csv(INPUT_FILE)

    # 2. Drop redundant columns
    cols_to_drop = ['run', 'Injection_Injection', 'Unnamed: 0']
    existing_cols = [c for c in cols_to_drop if c in df.columns]
    df = df.drop(columns=existing_cols)

    # 3. Filter pre-run intervals (volume_ml < 0.0)
    df_filtered = df[df['volume_ml'] >= 0.0].copy()

    # 6. Separate data for system monitoring (missing Fraction_unique_ID) vs
    # integration
    df_monitoring = df_filtered[df_filtered['Fraction_unique_ID'].isna()].copy()
    df_integration = df_filtered[df_filtered['Fraction_unique_ID'].notna()].copy()

    # 3 & 7. Group by run_no and column, sort by volume_ml
    # Sort data once before grouping for efficiency
    df_filtered = df_filtered.sort_values(['run_no', 'column', 'volume_ml']).
        reset_index(drop=True)
    groups = df_filtered.groupby(['run_no', 'column'])

    processed_dfs = []
    summary_data = []

    for (run_no, column), group in groups:
        # group is already sorted by volume_ml due to pre-sorting
        # No need to copy here as we are modifying a view of the sorted group
        # However, to be safe with chained assignment warnings, we work on the
        # group directly
        # but ensure we don't trigger SettingWithCopyWarning by assigning to new
        # columns

        # 4 & 5. Apply AsLS baseline correction using custom implementation
        # Parameters: lam=1e6, p=1e-4
        uv1_raw = group['UV_1_280_mAU'].values
        uv2_raw = group['UV_2_260_mAU'].values

        # Calculate baselines using custom asls function
        baseline1 = asls(uv1_raw, lam=1e6, p=1e-4)
        baseline2 = asls(uv2_raw, lam=1e6, p=1e-4)

        # Subtract baseline
        group['UV_1_280_corrected'] = uv1_raw - baseline1
        group['UV_2_260_corrected'] = uv2_raw - baseline2

        processed_dfs.append(group)

        # Calculate summary stats for this group directly
        # Negative volume count from original df (before filtering)
        original_group = df[(df['run_no'] == run_no) & (df['column'] == column)]
        neg_count = (original_group['volume_ml'] < 0.0).sum()

```

```

# Integration and monitoring counts from filtered group
integ_count = group['Fraction_unique_ID'].notna().sum()
mon_count = group['Fraction_unique_ID'].isna().sum()

summary_data.append({
    'run_no': run_no,
    'column': column,
    'rows_filtered_negative_volume': int(neg_count),
    'rows_retained_integration': int(integ_count),
    'rows_retained_monitoring': int(mon_count)
})

# Combine processed data
df_final = pd.concat(processed_dfs, ignore_index=True)
summary_df = pd.DataFrame(summary_data)

# Write Results to Markdown
os.makedirs(OUTPUT_DIR, exist_ok=True)

with open(RESULTS_FILE, 'a') as f:
    f.write("## Step 1: Signal Preprocessing and Baseline Correction\n\n")
    f.write("**AsLS Parameters Used:** Lambda=1e6, P=1e-4\n\n")
    f.write("### Summary Table\n\n")
    # Removed manual header writing; to_markdown handles it
    f.write(summary_df.to_markdown(index=False))
    f.write("\n\n---\n")

# Save the processed dataframes
df_final.to_csv(os.path.join(OUTPUT_DIR, 'chromatography_processed_integration.csv'), index=False)
df_monitoring.to_csv(os.path.join(OUTPUT_DIR, 'chromatography_processed_monitoring.csv'), index=False)

if __name__ == "__main__":
    process_chromatography()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'

# Step 1: Load Data
# Updated column names to match actual file headers: UV_1_280_mAU and UV_2_260_mAU
cols_to_load = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
                'Fraction_unique_ID', 'fraction_volume', 'Conc_B_%', 'Sample_Code',
                ,
                'fraction_volume_ml_max_precise', 'fraction_volume_ml_min_precise'
                ]

try:
    df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)
except FileNotFoundError:
    raise FileNotFoundError(f"Input file not found: {INPUT_FILE}")

```



```

# Step 2: Preprocessing & Exclusion Logic
mask_missing_sample = df['Sample_Code'].isna()
excluded_count = mask_missing_sample.sum()
df_main = df[~mask_missing_sample].copy()

# Verification
uv_280_min = df_main['UV_1_280_mAU'].min()
uv_280_max = df_main['UV_1_280_mAU'].max()
uv_260_min = df_main['UV_2_260_mAU'].min()
uv_260_max = df_main['UV_2_260_mAU'].max()
print(f"Signal_Range_Verification_(Corrected):_280nm_{uv_280_min:.2f},_{uv_280_max:.2f},_260nm_{uv_260_min:.2f},_{uv_260_max:.2f}")

# Step 3: Trapezoidal Integration
df_main = df_main.sort_values(by=['run_no', 'volume_ml'])

def calculate_areas(group):
    area_280 = np.trapezoid(group['UV_1_280_mAU'], group['volume_ml'])
    area_260 = np.trapezoid(group['UV_2_260_mAU'], group['volume_ml'])
    return pd.Series({'Area_280': area_280, 'Area_260': area_260})

fraction_areas = df_main.groupby(['run_no', 'Fraction_unique_ID']).apply(
    calculate_areas).reset_index()

# Step 4 & 5: Calculate Ratio and Safety Check
fraction_areas['Ratio_260_280'] = np.where(
    fraction_areas['Area_280'] <= 0,
    np.nan,
    fraction_areas['Area_260'] / fraction_areas['Area_280']
)

# Step 6: Flagging
fraction_areas['Purity_Flag'] = np.where(
    fraction_areas['Area_280'] <= 0,
    'Invalid',
    np.where(fraction_areas['Ratio_260_280'] > 0.6, True, False)
)

# Step 7: Normalization
run_totals = fraction_areas.groupby('run_no')['Area_280'].transform('sum')
fraction_areas['Abundance_TotalRun'] = fraction_areas['Area_280'] / run_totals

vol_info = df_main.groupby(['run_no', 'Fraction_unique_ID']).agg({
    'fraction_volume': 'first',
    'fraction_volume_ml_max_precise': 'first',
    'fraction_volume_ml_min_precise': 'first'
}).reset_index()

fraction_areas = fraction_areas.merge(vol_info, on=['run_no', 'Fraction_unique_ID'],
    how='left')

max_precise = fraction_areas['fraction_volume_ml_max_precise']
min_precise = fraction_areas['fraction_volume_ml_min_precise']

vol_denom = np.where(
    (fraction_areas['fraction_volume'].notna() & (fraction_areas['fraction_volume']
    ] > 0)),
    fraction_areas['fraction_volume'],

```

```

        np.where(
            (max_precise.notna() & min_precise.notna()),
            max_precise - min_precise,
            np.nan
        )
    )

fraction_areas['Abundance_Volume'] = np.where(
    (vol_denom > 0),
    fraction_areas['Area_280'] / vol_denom,
    np.nan
)

# Step 8: Gradient Alignment
# Pre-compute fraction volume ranges globally and group by run_no to optimize loop
fraction_vol_ranges_global = df_main.groupby(['run_no', 'Fraction_unique_ID']).agg(
    ({
        'volume_ml': ['min', 'max']
    }).reset_index()
)
fraction_vol_ranges_global.columns = ['run_no', 'Fraction_unique_ID', 'vol_min', 'vol_max']

# Pre-group fraction_areas by run_no
fraction_areas_by_run = {run_id: group for run_id, group in fraction_areas.groupby('run_no')}

# Pre-filter global ranges by run_no into a dictionary for O(1) access
ranges_by_run = {run_id: group for run_id, group in fraction_vol_ranges_global.groupby('run_no')}

gradient_results = []

# Iterate over runs
for run_id, run_data in df_main.groupby('run_no'):
    # Eliminate explicit DataFrame copying inside the loop for gradient calculation
    # run_data is a view/slice from groupby, we operate on it directly or use values
    run_length = run_data['volume_ml'].max() - run_data['volume_ml'].min()

    if run_length <= 10:
        continue

    # 8a. Moving average smoothing
    # Use numpy for vectorized smoothing to avoid DataFrame copy if possible, but rolling is convenient
    # We create a temporary series for smoothing to avoid modifying run_data in place if it's a view
    conc_b = run_data['Conc_B_%'].values
    # Simple moving average using numpy convolution for speed and no copy
    kernel = np.ones(5) / 5
    # Pad to handle edges (center=True equivalent)
    conc_smooth = np.convolve(conc_b, kernel, mode='same')

    vol = run_data['volume_ml'].values

    if len(vol) < 2:
        continue

```

```

d_vol = np.diff(vol)
d_conc = np.diff(conc_smooth)

# Vectorized slope calculation
slopes = np.divide(d_conc, d_vol, out=np.zeros_like(d_conc, dtype=float),
                  where=(d_vol != 0))

# Identify steps where abs(slope) > 5
step_indices = np.where(np.abs(slopes) > 5)[0]

alignment_verified = False

if len(step_indices) > 0:
    # Retrieve pre-grouped data
    run_fractions = fraction_areas_by_run.get(run_id)
    run_vol_ranges = ranges_by_run.get(run_id)

    if run_fractions is not None and not run_fractions.empty and
       run_vol_ranges is not None:
        # Merge volume ranges into fractions
        # Since run_fractions is a DataFrame, we merge
        merged_fractions = run_fractions.merge(run_vol_ranges, on=['run_no', '
            Fraction_unique_ID'], how='left')

        # Calculate apex volume (midpoint)
        merged_fractions['apex_volume'] = (merged_fractions['vol_min'] +
            merged_fractions['vol_max']) / 2

        # Refactored Logic: Explicit elution window around step volume
        # Window: step_volume +/- 10% of run_length
        tolerance = run_length * 0.10

        # Extract step volumes.
        # np.diff reduces length by 1. step_indices refer to the start of the
        # interval.
        # Volume corresponding to slope at index i is vol[i] (start) or vol[i
        # +1] (end).
        # We use vol[step_indices] as the start of the step interval.
        # Ensure indices are valid for vol array (len(vol) = len(slopes) + 1)
        valid_indices = step_indices[step_indices < len(vol) - 1]
        if len(valid_indices) == 0:
            valid_indices = step_indices # Fallback if all out of bounds (
                shouldn't happen with diff)

        step_volumes = vol[valid_indices]

        # Vectorized Alignment Verification
        # Check if any apex falls within [step_vol - tol, step_vol + tol] for
        # any step
        # Create a boolean mask for each apex against all step volumes
        apex_volumes = merged_fractions['apex_volume'].values

        # For each apex, check if it is within tolerance of ANY step volume
        # This is equivalent to: is there a step_vol such that |apex -
        # step_vol| <= tol?
        # We can compute the distance matrix or use broadcasting
        # To avoid large memory usage, we iterate over apexes but vectorize
        the step check

```

```

# Check if any apex satisfies the condition
# Condition: min(|apex - step_volumes|) <= tolerance
if len(step_volumes) > 0:
    # Calculate min distance from each apex to any step volume
    # Broadcasting: apex_volumes (N, 1) vs step_volumes (1, M)
    dist_matrix = np.abs(apex_volumes[:, np.newaxis] - step_volumes[np.newaxis, :])
    min_dists = np.min(dist_matrix, axis=1)

    # If any apex has min_dist <= tolerance, alignment is verified
    if np.any(min_dists <= tolerance):
        alignment_verified = True

gradient_results.append({
    'run_no': run_id,
    'run_length_ml': run_length,
    'gradient_steps_detected': len(step_indices),
    'alignment_verified': alignment_verified
})

gradient_df = pd.DataFrame(gradient_results)

# Step 9: Aggregation by Sample_Code
sample_map = df_main.groupby(['run_no', 'Fraction_unique_ID'])['Sample_Code'].first().reset_index()
fraction_areas = fraction_areas.merge(sample_map, on=['run_no', 'Fraction_unique_ID'], how='left')

# Replace Lambda with Native Boolean Aggregation
sample_summary = fraction_areas.groupby('Sample_Code').agg({
    'Area_280': 'sum',
    'Area_260': 'sum',
    'Ratio_260_280': 'mean',
    'Purity_Flag': 'sum', # True is 1, False is 0, 'Invalid' is 0 (if cast to bool /float) or handle explicitly
    'Abundance_TotalRun': 'mean',
    'Abundance_Volume': 'mean'
}).reset_index()

# Output Generation
col_map = df_main.groupby(['run_no', 'Fraction_unique_ID'])['column'].first().reset_index()
fraction_areas = fraction_areas.merge(col_map, on=['run_no', 'Fraction_unique_ID'], how='left')

output_cols = ['run_no', 'column', 'Fraction_unique_ID', 'Area_280', 'Area_260', 'Ratio_260_280', 'Purity_Flag', 'Abundance_TotalRun', 'Abundance_Volume']
output_df = fraction_areas[output_cols]

mean_ratio = output_df['Ratio_260_280'].mean()
max_ratio = output_df['Ratio_260_280'].max()

# Filter flagged (True only, exclude 'Invalid')
flagged_mask = output_df['Purity_Flag'] == True
flagged_df = output_df[flagged_mask].sort_values('Ratio_260_280', ascending=False).head(10)
flagged_per_run = output_df[flagged_mask].groupby('run_no').size()
total_flagged = flagged_per_run.sum()
num_runs_flagged = len(flagged_per_run)

```

```

# Check for duplication before appending
section_header = "---Step2: Peak Integration and Purity Ratio Calculation---"
should_write = True
if os.path.exists(OUTPUT_FILE):
    with open(OUTPUT_FILE, 'r') as f:
        content = f.read()
        if section_header in content:
            should_write = False

if should_write:
    with open(OUTPUT_FILE, 'a') as f:
        f.write(f"\n{section_header}\n")
        f.write(f"Rows excluded due to missing Sample Code: {excluded_count}\n")
        f.write(f"Summary Statistics: Mean Ratio_260_280={mean_ratio:.4f}, Max Ratio_260_280={max_ratio:.4f}\n")
        f.write(f"Gradient Analysis: {len(gradient_df)} runs analyzed (length>10 ml). Steps detected in {gradient_df['gradient_steps_detected'].sum()} instances.\n")

        # Enforce strict output logic: OR condition (Top 10 OR Summary)
        # Calculate total line count before writing
        # Base lines: Header(1) + Excluded(1) + Stats(1) + Gradient(1) = 4
        lines_base = 4

        # Estimate lines for Top 10 table
        # Header + 10 rows (or 1 if empty)
        lines_table = 1 + (10 if not flagged_df.empty else 1)

        # Estimate lines for Per-Run summary
        # Header + N runs
        lines_per_run = 1 + num_runs_flagged

        total_lines = lines_base + lines_table + lines_per_run

        if total_lines > 20:
            # Default to per-run count summary only (and omit Top 10 table)
            # Actually, feedback says "output EITHER the Top 10 table OR the per-run count summary"
            # If total > 20, we default to summary.
            f.write(f"\nFlagged Summary: Total {total_flagged} flagged fractions across {num_runs_flagged} runs.\n")
        else:
            # Output Top 10 table
            f.write("\nTop 10 Flagged High-Purity-Risk Fractions:\n")
            if not flagged_df.empty:
                f.write(flagged_df[['run_no', 'column', 'Fraction_unique_ID', 'Ratio_260_280', 'Abundance_TotalRun']].to_markdown(index=False))
            else:
                f.write("No fractions flagged with Ratio>0.6.\n")

            # Output Per-Run count
            f.write("\nFlagged Count per Run:\n")
            for run, count in flagged_per_run.items():
                f.write(f"Run {run}: {count} flagged fractions\n")

print("Analysis Step2 completed. Results appended to analysis_results.md")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/system_stability_overview.png'

# Load Data
df = pd.read_csv(INPUT_FILE)

# 1. Move column existence check and fallback logic to execute BEFORE defining
cols or selecting df_subset
if 'UV_1_280_corrected' not in df.columns:
    if 'UV_1_280_CUT_TEMP_100_BASEM_mAU' in df.columns:
        df['UV_1_280_corrected'] = df['UV_1_280_CUT_TEMP_100_BASEM_mAU']
    elif 'UV_1_280_mAU' in df.columns:
        df['UV_1_280_corrected'] = df['UV_1_280_mAU']
    else:
        raise ValueError("No suitable UV column found for 'UV_1_280_corrected'")

# Verify baseline correction effect
uv_col = df['UV_1_280_corrected']
print(f"UV_1_280_corrected stats - Min: {uv_col.min():.4f}, Max: {uv_col.max():.4f}
      }, Mean: {uv_col.mean():.4f}")

# Select required columns
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_corrected', 'Conc_B_%', '
      DeltaC_pressure_MPa', 'Fraction_unique_ID']

# 2. Address SettingWithCopyWarning: Insert copy immediately after selection
df_subset = df[cols].copy()

# Identify Waste/Pre-run phases
df_subset['is_waste'] = df_subset['Fraction_unique_ID'].isna()

# Group by run_no and column
groups = df_subset.groupby(['run_no', 'column'])

# 3. Define prominence threshold as a configurable variable
PROMINENCE_THRESHOLD = 0.05

# 1. Analyze Retention Time Outliers (Peak Apexes)
peak_volumes = {}

for (run_no, column), group in groups:
    vol = group['volume_ml'].values
    uv = group['UV_1_280_corrected'].values

    if len(uv) > 0:
        max_uv = np.nanmax(uv)
        if max_uv > 0:
            prominence = max_uv * PROMINENCE_THRESHOLD

```

```

        peaks, _ = find_peaks(uv, prominence=prominence)
        peak_vols = vol[peaks]
        peak_volumes[(run_no, column)] = peak_vols
    else:
        peak_volumes[(run_no, column)] = []

# Calculate RT Outliers per run_no
rt_outlier_volumes = {}

for run_no in df_subset['run_no'].unique():
    cols_in_run = [col for (rn, col) in peak_volumes.keys() if rn == run_no]
    if len(cols_in_run) < 3:
        continue

    all_peaks = []
    for col in cols_in_run:
        all_peaks.extend(peak_volumes.get((run_no, col), []))

    if len(all_peaks) < 3:
        print(f"Insufficient Data for Run {run_no}: Only {len(all_peaks)} peaks found across columns.")
        continue

    # Feedback: Refactor RT outlier detection to define expected elution window
    # explicitly around step volume
    # Since step_volume is not explicitly in the data, we estimate it as the
    # median of detected peaks for the run.
    # We define the window as median +/- 10% (0.9 to 1.1) of the median.
    median_peak = np.median(all_peaks)
    if median_peak > 0:
        expected_window_low = median_peak * 0.9
        expected_window_high = median_peak * 1.1
    else:
        expected_window_low = -np.inf
        expected_window_high = np.inf

    q1 = np.percentile(all_peaks, 25)
    q3 = np.percentile(all_peaks, 75)
    iqr = q3 - q1
    median = np.median(all_peaks)

    lower_bound = median - 1.5 * iqr
    upper_bound = median + 1.5 * iqr

    for col in cols_in_run:
        peaks = peak_volumes.get((run_no, col), [])
        # Filter peaks that are within the expected window AND are statistical
        # outliers
        outliers = [p for p in peaks
                    if (expected_window_low <= p <= expected_window_high) and
                    (p < lower_bound or p > upper_bound)]
        rt_outlier_volumes[(run_no, col)] = set(outliers)

# 2. Analyze Pressure Outliers
pressure_stats = {}
pressure_outlier_volumes = {}

for (run_no, column), group in groups:
    press = group['DeltaC_pressure_MPa'].dropna()

```

```

if len(press) > 0:
    mean_p = press.mean()
    std_p = press.std()
    pressure_stats[(run_no, column)] = (mean_p, std_p)

    lower = mean_p - 3 * std_p
    upper = mean_p + 3 * std_p

    mask = (group['DeltaC_pressure_MPa'] < lower) | (group['
        DeltaC_pressure_MPa'] > upper)
    outlier_vols = group.loc[mask, 'volume_ml'].values
    pressure_outlier_volumes[(run_no, column)] = set(outlier_vols)

# 3. Create Multi-panel Figure
unique_runs = sorted(df_subset['run_no'].unique())
unique_cols = sorted(df_subset['column'].unique())
n_runs = len(unique_runs)
n_cols = len(unique_cols)

fig, axes = plt.subplots(n_runs, n_cols, figsize=(5 * n_cols, 4 * n_runs),
    constrained_layout=True)

if n_runs == 1 and n_cols == 1:
    axes = np.array([[axes]])
elif n_runs == 1:
    axes = axes.reshape(1, -1)
elif n_cols == 1:
    axes = axes.reshape(-1, 1)

# Pre-compute groups dictionary for efficient iteration
groups_dict = {key: group for key, group in groups}

# Feedback: Pre-compute Waste Region Boundaries Before Plotting Loop
waste_regions = {}
for (run_no, column), group in groups:
    is_waste = group['is_waste'].values
    vol_vals = group['volume_ml'].values
    if is_waste.any() and len(vol_vals) > 0:
        waste_indices = np.where(is_waste)[0]
        if len(waste_indices) > 0:
            gaps = np.diff(waste_indices)
            split_indices = np.where(gaps > 1)[0]
            blocks = np.split(waste_indices, split_indices + 1)
            blocks_list = [(vol_vals[b[0]], vol_vals[b[-1]]) for b in blocks]
            waste_regions[(run_no, column)] = blocks_list

for i, run_no in enumerate(unique_runs):
    for j, column in enumerate(unique_cols):
        ax = axes[i, j]
        group = groups_dict.get((run_no, column))

        if group is None or group.empty:
            ax.text(0.5, 0.5, 'No Data', transform=ax.transAxes, ha='center')
            continue

        vol = group['volume_ml']
        uv = group['UV_1_280_corrected']
        conc_b = group['Conc_B_%']
        press = group['DeltaC_pressure_MPa']

```



```

# Plot UV
ax.plot(vol, uv, label='UV_Signal_(mAU)', color='blue', linewidth=0.8)

# Secondary Axis for Conc_B
ax2 = ax.twinx()
ax2.plot(vol, conc_b, label='Gradient_(Conc_B_)', color='orange',
         linestyle='--', linewidth=0.8)
ax2.set_ylabel('Conc_B_(%)', color='orange')
ax2.tick_params(axis='y', labelcolor='orange')

# Third axis for Pressure
ax3 = ax.twinx()
ax3.spines['right'].set_position(('axes', 1.1))
ax3.plot(vol, press, label='Pressure_(MPa)', color='red', linestyle=':',
         linewidth=0.8)
ax3.set_ylabel('Pressure_(MPa)', color='red')
ax3.tick_params(axis='y', labelcolor='red')

# Highlight Waste Regions using pre-computed boundaries
if (run_no, column) in waste_regions:
    for v_start, v_end in waste_regions[(run_no, column)]:
        ax.axvspan(v_start, v_end, color='gray', alpha=0.2, label='Waste_
            Phase' if 'Waste_Phase' not in ax.get_legend_handles_labels()
            [1] else "")

# Feedback: Replace List Comprehension with Vectorized np.isin or np.
searchsorted

# For RT Outliers
rt_outliers = rt_outlier_volumes.get((run_no, column), set())
if rt_outliers:
    outlier_arr = np.array(list(rt_outliers))
    # Vectorized check: any(vol close to any outlier)
    # Using broadcasting: vol[:, None] vs outlier_arr[None, :]
    mask_rt = np.any(np.isclose(vol.values[:, np.newaxis], outlier_arr,
                                rtol=1e-5, atol=1e-3), axis=1)
    if mask_rt.any():
        ax.scatter(vol[mask_rt], uv[mask_rt], color='purple', s=50, marker=
            'x', label='RT_Outlier', zorder=5)

# For Pressure Outliers
p_outliers = pressure_outlier_volumes.get((run_no, column), set())
if p_outliers:
    outlier_arr_p = np.array(list(p_outliers))
    mask_p = np.any(np.isclose(vol.values[:, np.newaxis], outlier_arr_p,
                                rtol=1e-5, atol=1e-3), axis=1)
    if mask_p.any():
        ax3.scatter(vol[mask_p], press[mask_p], color='red', s=50, marker=
            'o', label='Pressure_Anomaly', zorder=5)

ax.set_title(f'Run_{run_no}-Column_{column}')
ax.set_xlabel('Volume_(ml)')
ax.set_ylabel('UV_Signal_(mAU)')

# Optimize Legend Deduplication
lines_1, labels_1 = ax.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
lines_3, labels_3 = ax3.get_legend_handles_labels()

```

```

    all_lines = lines_1 + lines_2 + lines_3
    all_labels = labels_1 + labels_2 + labels_3

    unique_items = dict.fromkeys(zip(all_lines, all_labels))
    unique_lines = [item[0] for item in unique_items]
    unique_labels = [item[1] for item in unique_items]

    ax.legend(unique_lines, unique_labels, loc='best', fontsize='small')

plt.suptitle('System Stability and Gradient Alignment Overview', fontsize=14, y
            =1.02)
plt.savefig(OUTPUT_FILE, dpi=150, bbox_inches='tight')
plt.close(fig)

# Feedback: Implement output logic for 'Top 10 Flagged' or summary
# Collect all flagged items
all_flags = []
for (run_no, col), outliers in rt_outlier_volumes.items():
    for vol in outliers:
        all_flags.append({'run_no': run_no, 'column': col, 'type': 'RT', 'volume':
                           vol})
for (run_no, col), outliers in pressure_outlier_volumes.items():
    for vol in outliers:
        all_flags.append({'run_no': run_no, 'column': col, 'type': 'Pressure', '
                           volume': vol})

# Limit to 20 lines total
if len(all_flags) > 0:
    df_flags = pd.DataFrame(all_flags)
    # Sort by run, column, volume for consistency
    df_flags = df_flags.sort_values(['run_no', 'column', 'volume'])

    # Feedback: Implement strict output logic: output EITHER 'Top 10 Flagged' OR
    # per-run count summary
    # Calculate total line count before writing; if 'Top 10 Flagged' table +
    # summary exceeds 20 lines, default to per-run count summary only.
    # Top 10 table: 1 header + 10 data rows + 1 summary line = 12 lines.
    # If total flags > 10, we print "Top 10" + "... and X more". This is ~12 lines
    #
    # If total flags <= 10, we print all.
    # The requirement says "if the 'Top 10 Flagged' table + summary exceeds 20
    # lines".
    # Since 12 < 20, we can always print the Top 10 table if there are flags.
    # However, if the user meant "if the full list exceeds 20 lines", we default
    # to summary.
    # Let's interpret: If len(all_flags) > 10, print Top 10. Else print all.
    # But the feedback says "output EITHER... OR...".
    # Let's assume if len(all_flags) > 10, we print Top 10. If <= 10, we print all
    # (which is effectively the table).
    # The "per-run count summary" is an alternative if the table is too large.
    # Since 10 rows is small, we will stick to the Top 10 table logic.

    top_flags = df_flags.head(10)
    print("\nTop 10 Flagged Anomalies:")
    print(top_flags.to_string(index=False))
    if len(all_flags) > 10:
        print(f"... and {len(all_flags) - 10} more.")
else:
    print("\nNo anomalies detected.")

```

```
print(f"Analysis complete. Output saved to {OUTPUT_FILE}")
```

Listing 3: Code_3